Practices 6&7. Search, Insertion, Deletion, and Tree Traversals

[CSE2010] Data Structures

Department of Data Science

Practice 4 Solution in C++

- Function isPalindrome using a stack
  - There exists a solution without using a stack
- Function balance uses a stack.

```
// Return true if str is a palindrome; false otherwise
bool isPalindrome(string& str) {
  stack<char> s;
  for (int i = 0; i < str.length(); ++i)
    s.push(str.at(i));
  for (int i = 0; i < str.length(); ++i) {
    char e = s.top();
    s.pop();
    if (e != str.at(i))
      return false;
  }
  return true;
}
```

```
char match(char ch) {
  switch(ch) {
    case ')':
      return '(';
    case ']':
      return '[';
    case '}':
      return '{';
    default:
      cerr<<"match: invalid input"<<endl;
      exit(1);
  }
}

// Return true if brackets are balanced in str; false otherwise
bool balance(string& str) {
  stack<char> s;
  string opening = "([{";
  string closing = ")]}";
  for (int i = 0; i < str.length(); ++i) {
    char ch = str.at(i);
    if (opening.find(ch) != string::npos)
      s.push(ch);
    else if (closing.find(ch) != string::npos) {
      if (s.empty() || s.top() != match(ch))
        return false;
      s.pop();
    }
  }
  return s.empty();
}
```

Practice 4 Solution in Python

- Function isPalindrome using a stack
  - There exists a solution without using a stack
- Function balance uses a stack.

```python
# Return True if str is a palindrome; False otherwise
def isPalindrome(string):
  stack = []
  for s in string:
    stack.append(s)
  for s in string:
    if stack.pop() != s:
      return False
  return True
```

```python
# Return true if brackets are balanced in str; false otherwise
def balance(string):
  opening = ['(', '[', '{']
  closing = [')', ']', '}']
  match = {}
  for i, ch in enumerate(closing):
    match[ch] = opening[i]
  stack = []
  for ch in string:
    if ch in opening:
      stack.append(ch)
    elif ch in closing:
      if len(stack) == 0 or stack[-1] != match[ch]:
        return False
      stack.pop()
  return len(stack) == 0
```

Overview of Practices 6&7

- Implement **search** and tree **traversals** in a binary search tree (BST).
- Functions (TODO)
  1. **Search** for a given key in a BST
  2. **Inorder** traversal
  3. **Preorder** traversal
  4. **Postorder** traversal

- Implement **insertion** and **deletion** of a binary search tree (BST).
- Functions (TODO)
  1. **Insert** a given integer key into BST
  2. **Delete** a given integer key from BST

- Add these operations to the implementation on BST for last practice.
- TODO. Write the **time complexity** for each of above functions in your implementation.

Input of BST

- Each line represents a single operation.
  1. **S<space>[int]**
     - If the integer key [int] exists, write that integer to output file.
     - Otherwise, immediately terminate the program with an error.
  2. **N**
     - Write a sequence of keys visited in inorder traversal.
  3. **R**
     - Write a sequence of keys visited in preorder traversal.
  4. **O**
     - Write a sequence of keys visited in postorder traversal.
  5. **I<space>[int]**
     - If [int] already exists in the tree, do nothing.
     - Otherwise, insert a new node with key [int], and **write "I<space>[int]" into output file.**
  6. **D<space>[int]**
     - If the deletion of [int] fails, immediately terminate the program with the error message.
     - Otherwise, delete the node with key [int], and **write "D<space>[int]" into output file.**

Input and Output

- Each line in output file represents the result of the corresponding line of the input file.
- Input File

```
B 11 23 36 48 51 59 63 71 86 92
S 23
N
R
O
```

Output File

```
B
23
11 23 36 48 51 59 63 71 86 92
51 23 11 36 48 71 59 63 86 92
11 48 36 23 63 59 92 86 71 51
```

- Input File

```
B 11 23 36 48 51 59 63 71 86 92
S 3
N
R
O
```

Output File

```
B
```

```
[hjkim@localhost bst]$ ./practice5 input8.txt output8.txt
                      51

            23                71

      11        36        59        86

                 48         63         92

[hjkim@localhost bst]$ cat input8.txt
B 11 23 36 48 51 59 63 71 86 92
S 23
N
R
O
[hjkim@localhost bst]$ cat output8.txt
B
23
11 23 36 48 51 59 63 71 86 92
51 23 11 36 48 71 59 63 86 92
11 48 36 23 63 59 92 86 71 51
```

```
[hjkim@localhost bst]$ ./practice5 input7.txt output7.txt
                      51

            23                71

      11        36        59        86

                 48         63         92

Search failed
[hjkim@localhost bst]$ cat input7.txt
B 11 23 36 48 51 59 63 71 86 92
S 3
N
R
O
[hjkim@localhost bst]$ cat output7.txt
B
```

Input and Output

- Each line in output file represents the result of the corresponding line of the input file.
- Input File & Output File

```
N
I 5
I 2
I 11
I 3
I 8
N
I 7
D 5
N
```

```

I 5
I 2
I 11
I 3
I 8
2 3 5 8 11
I 7
D 5
2 3 7 8 11
```

Input File & Output File

```
I 5
I 2
I 11
D 2
D 1
N
```

```
I 5
I 2
I 11
D 2
```

Running Example

- Start from the code on last practice.

```
[hjkim@localhost bst]$ make
g++ -std=c++11 -o practice5 bst.cpp
[hjkim@localhost bst]$ ./practice5 input5.txt output5.txt
[hjkim@localhost bst]$ cat input5.txt
N
I 5
I 2
I 11
I 3
I 8
N
I 7
D 5
N
[hjkim@localhost bst]$ cat output5.txt

I 5
I 2
I 11
I 3
I 8
2 3 5 8 11
I 7
D 5
2 3 7 8 11
[hjkim@localhost bst]$
```

```
[hjkim@localhost bst]$ ./practice5 input6.txt output6.txt
Deletion failed
[hjkim@localhost bst]$ cat input6.txt
I 5
I 2
I 11
D 2
D 1
N
[hjkim@localhost bst]$ cat output6.txt
I 5
I 2
I 11
D 2
[hjkim@localhost bst]$
```

Hints

- Implement recursive functions for search and traversals.
  - Assume that you are given **the root of a subtree** in the definition of a function that recursively traverse down the tree.
  - Call that function with **the root of the tree**.

- Make sure you defined a **base case** as well as a **recursive case** in a recursive function.
  - For the details on recursive functions, refer to the lecture slides on “abstract_data_type”.

```
int Fact(int n){
    if(n == 0) // Base case
        return 1;// Terminate
    else // Recursive case
        return n * Fact(n-1);
}
```

Submission Guideline & Announcement

- Submission: **source code, an input file, an output file**
  - NOTE: Write **time complexity** of each operation in your source code
  - Where: Assignments in LMS
  - Deadline: **23:59, April 21th (Sunday)**
- **Midterm exam on April 24th (Wednesday)**